

E-graphs modulo theories

presented by Sofia Brookie

E-graphs overview

Equality saturation

Handles non-ground equations (**identities**): $\forall x. f(x) = g(x, x)$



Congruence closure

Handles **ground** non-atomic equations: $f(e_1, e_2) = e_3$



Union-find

Handles **ground atomic** equations: $e_1 = e_2$

Equality saturation

If we have $\forall x..., t = s$, try looking for **instances** of this in the E-graph.

If we find an instance, *we might have to add the other side* into the E-graph, if it is not yet present. Then, we merge the corresponding E-classes.

E-graphs vs AC

Or really, equality saturation vs AC

A: associativity

$$f(x, f(y, z)) = f(f(x, y), z)$$

C: commutativity

$$f(x, y) = f(y, x)$$

AC: both

E-graphs vs AC

AC is the bane of equality saturation (or AC + other stuff, ACU, abelian groups, etc.).

We will use ‘+’ as our example AC-operator.

E-graphs vs AC

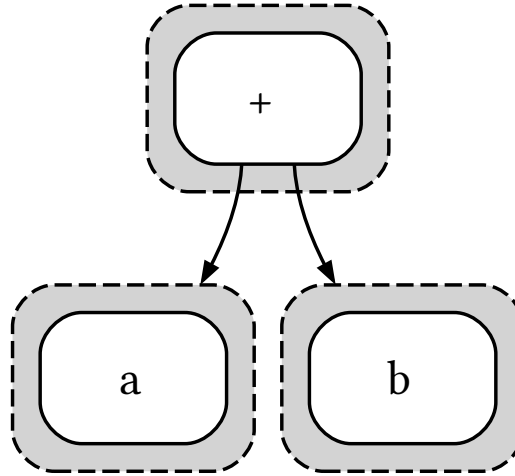


Figure 1: Size 2

E-graphs vs AC

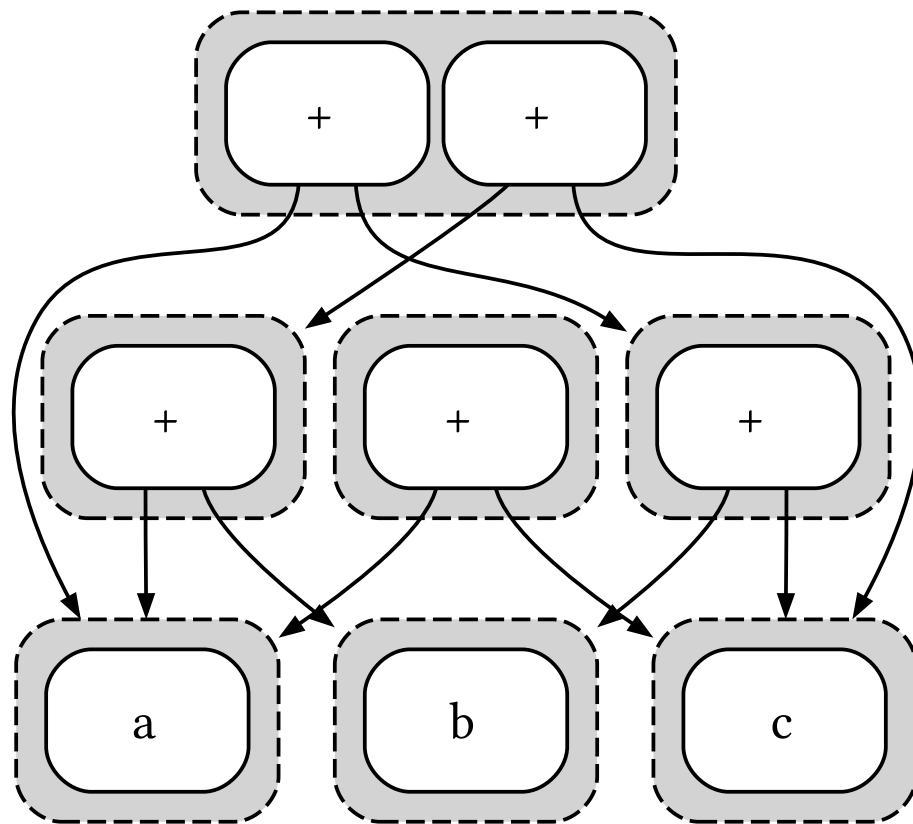


Figure 2: Size 3

E-graphs vs AC

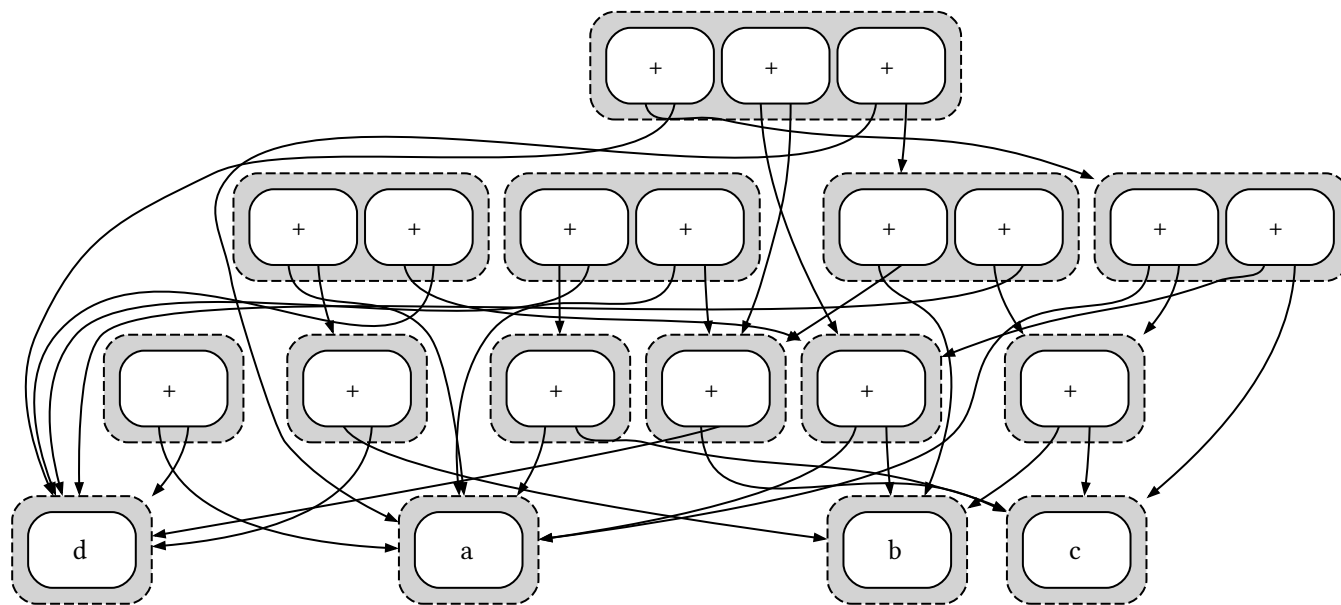


Figure 3: Size 4

E-graphs vs AC

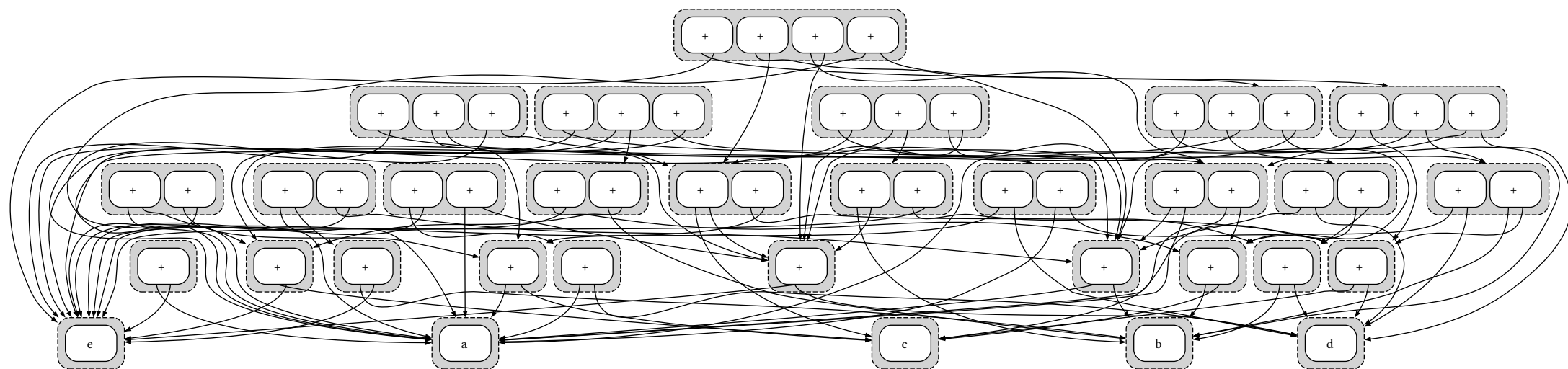


Figure 4: Size 5

E-graphs vs AC

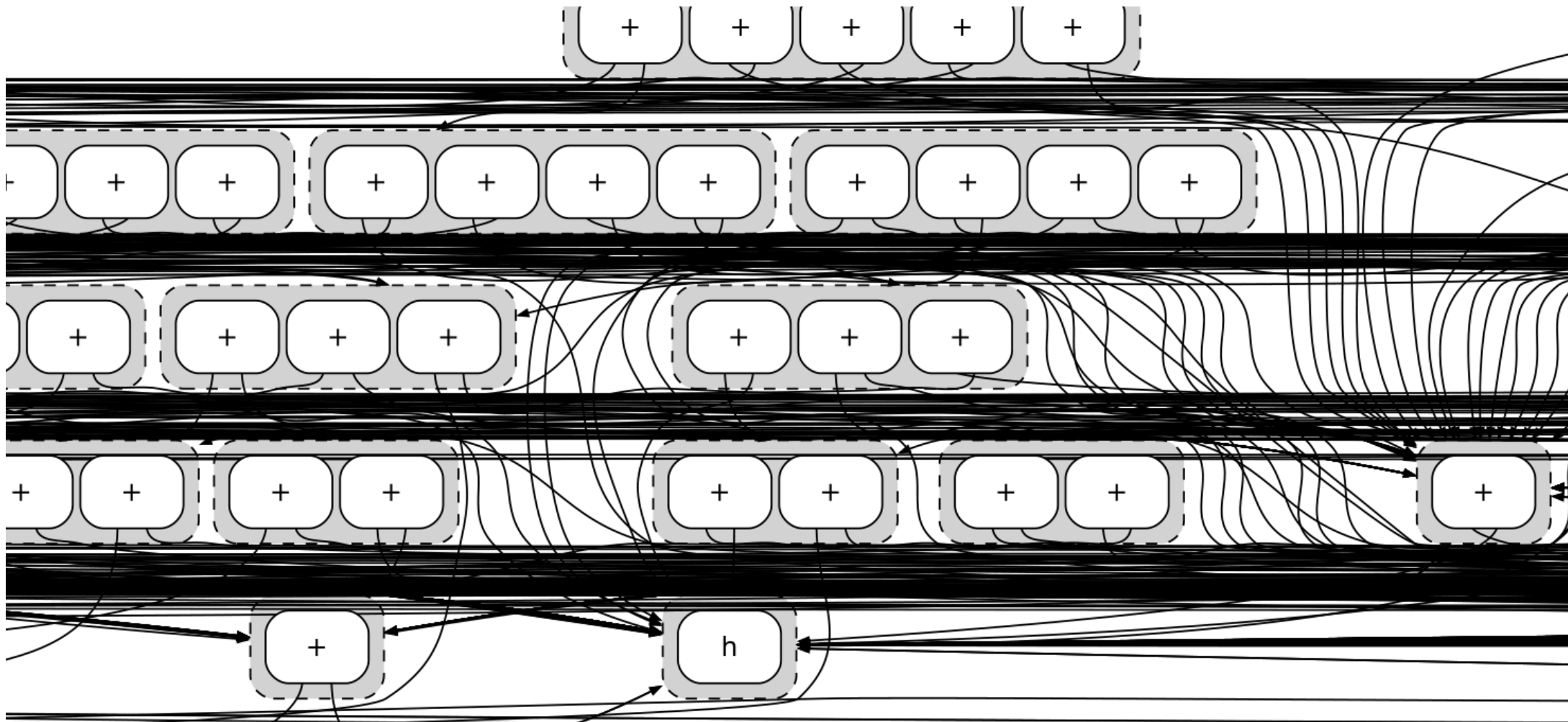


Figure 5: Size 8 (a small window)

E-graphs vs AC

Simply saturating with A and C gives $O(2^n)$ E-classes if you have a starting term with n +s ($a + b + c + \dots$). The total number of E-nodes across those classes is $O(n2^n)$.

(Non)-solution

Backoff methods: fire rules less and less if they are used too many times

(Non)-solution

Backoff methods: fire rules less and less if they are used too many times

Issues:

- Doesn't help if you run to saturation
- Can slow down finding the equations you want to find

AC problems, an example

Consider an affine function f with identity

$$\forall xy. f(x + y) + 1 = f(x) + f(y)$$

and the starting term $f(a) + f(b) + f(c)$. We can to rewrite this to $f(a + b + c) + 2$

E-graphs vs AC

Using the online Egglog demo:

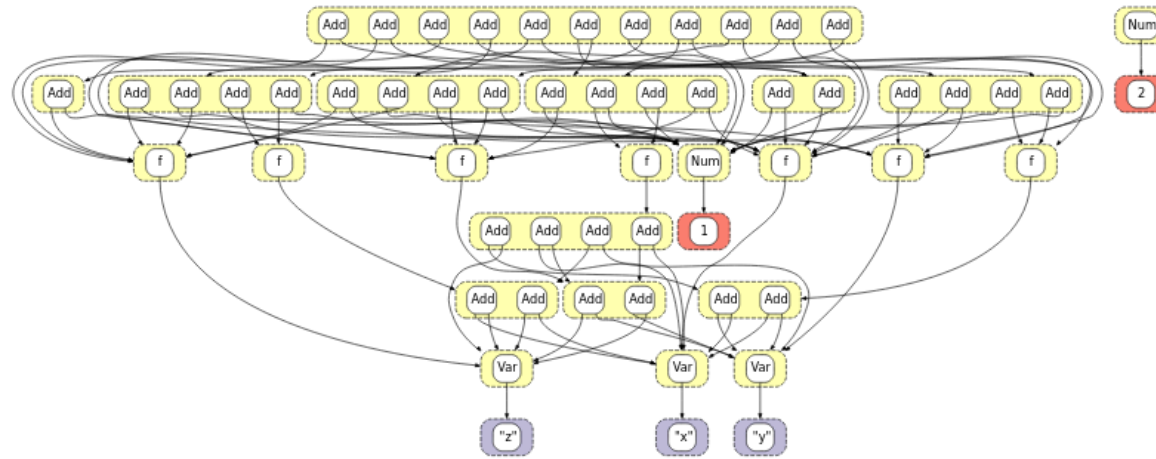


Figure 6: E-graph at timeout

The solution? EMTs

The SAT people have added theory-specific solvers and made very successful SMT solvers. Why can't we handle AC in a specific way, rather than relying on equality saturation?

The solution? EMTs

The SAT people have added theory-specific solvers and made very successful SMT solvers. Why can't we handle AC in a specific way, rather than relying on equality saturation?

This is a natural idea, that has been thrown around for a while.

EMT example

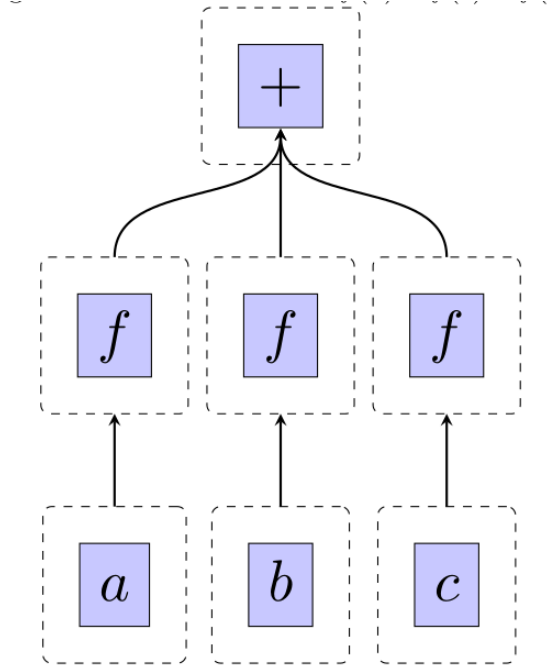


Figure 7: EMTs to the rescue...

EMT example

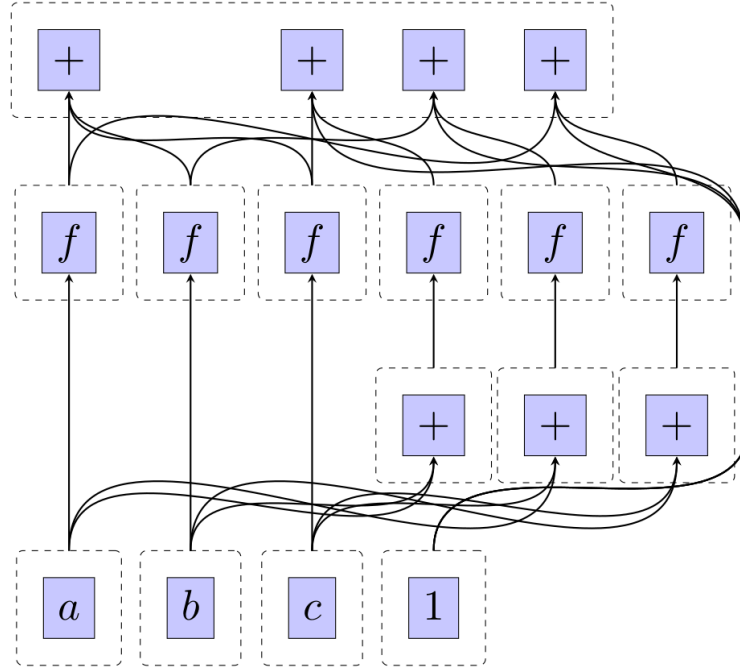


Figure 8: EMTs to the rescue...

The solution??

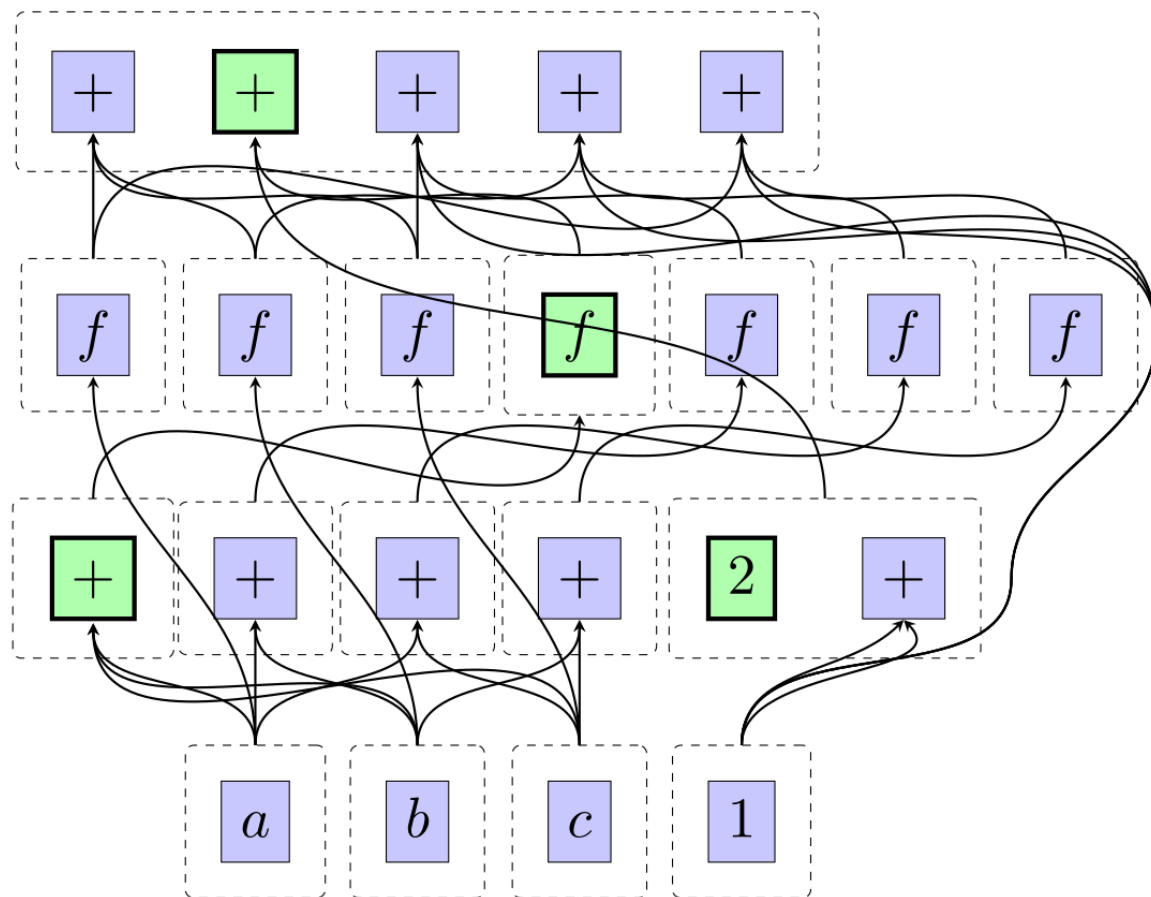


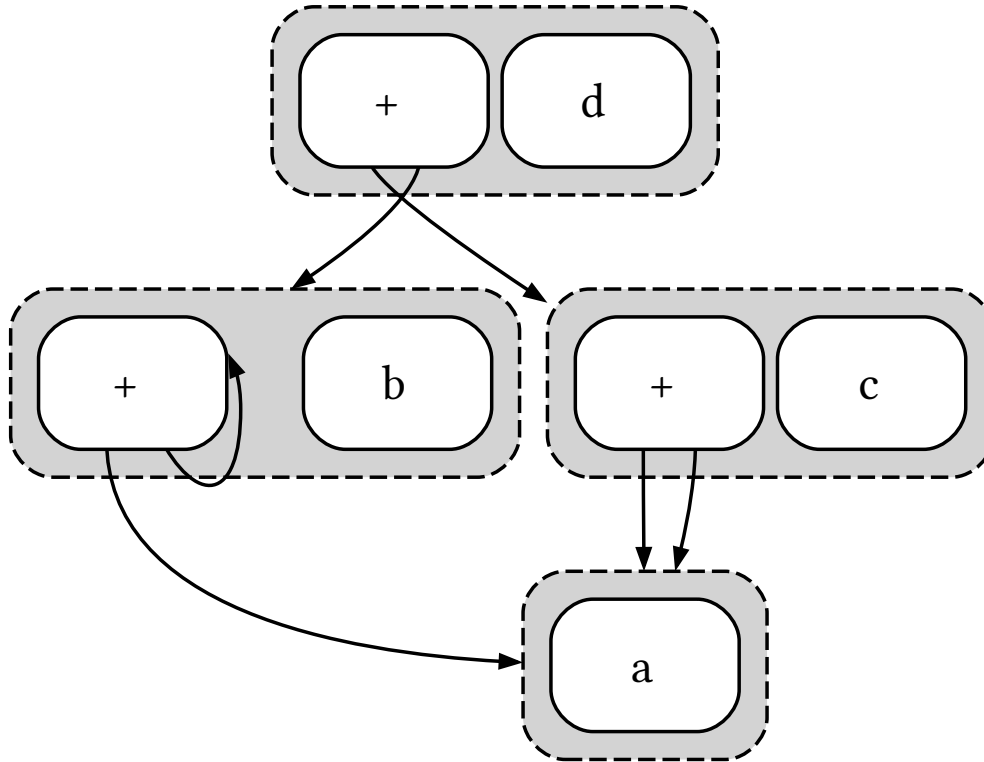
Figure 9: EMTs to the rescue...

Gaps

$$a + b = b$$

$$a + a = c$$

$$c + b = d$$



Gaps

$$(a + b = b)$$

$$(a + a = c)$$

$$(c + b = d)$$

b

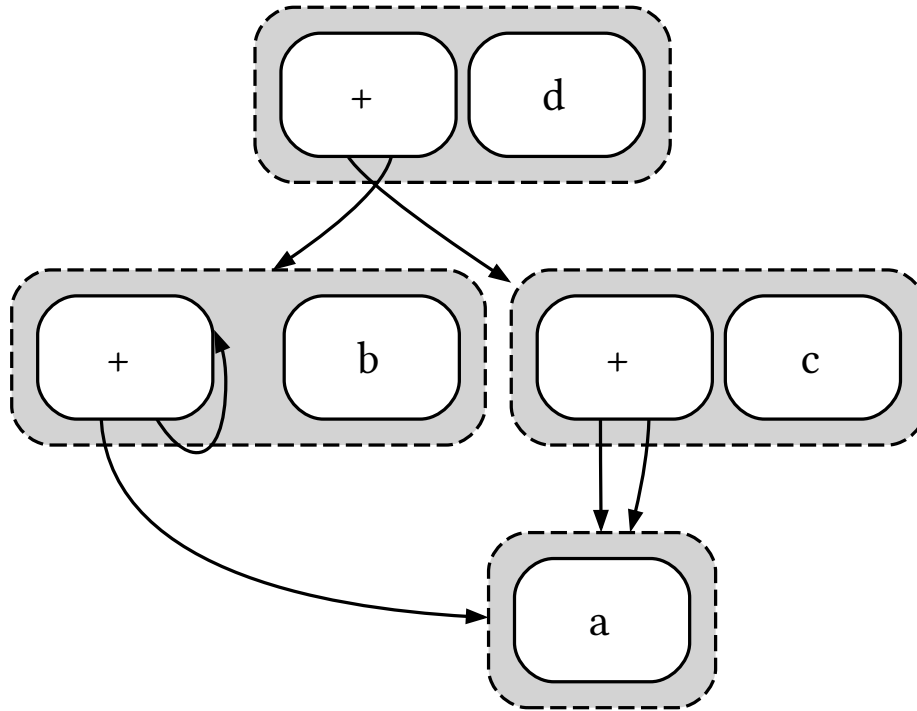
$$= a + b$$

$$= a + (a + b)$$

$$= (a + a) + b$$

$$= c + b$$

$$= d$$



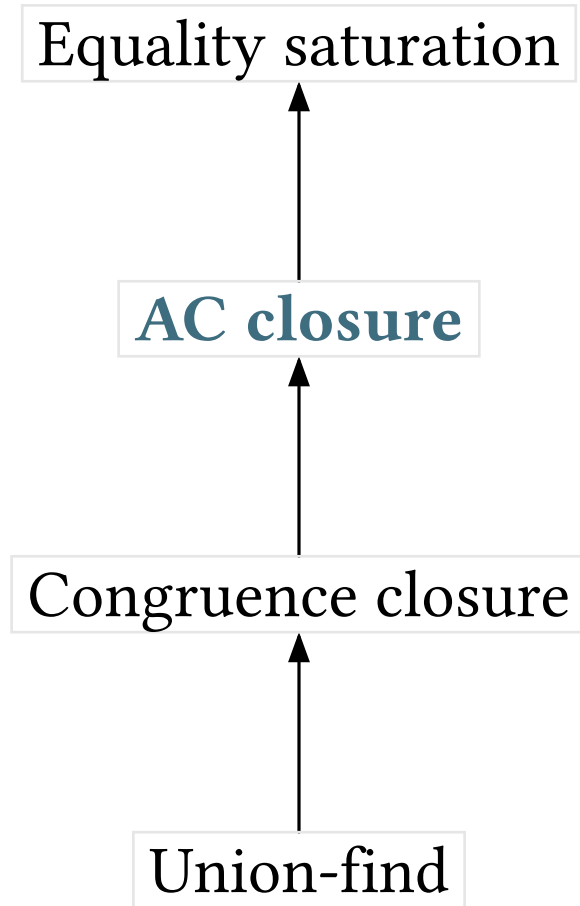
The problem

Just making nodes work “up to AC” has lost the equational reasoning required to properly handle AC.

We have an E-graph closed under congruence, but it’s missing the desired equation.

We could add back saturation on the AC identities, but that unsolves the problem we were trying to solve in the first place.

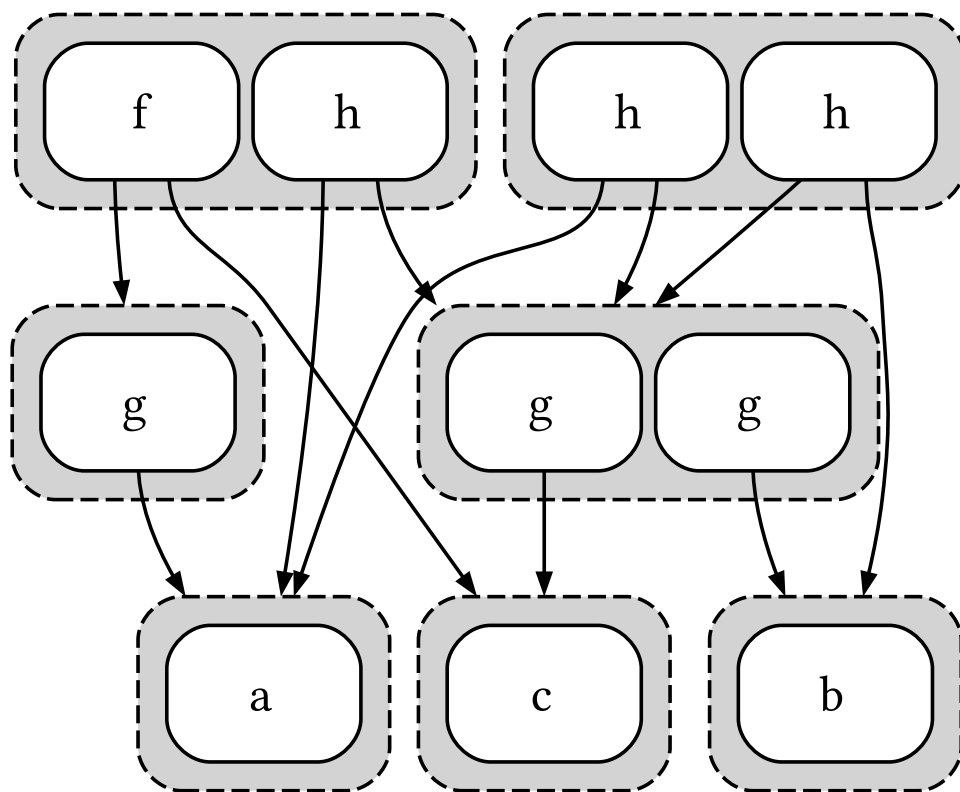
Making EMTs work: our contribution



Some insights from rewriting

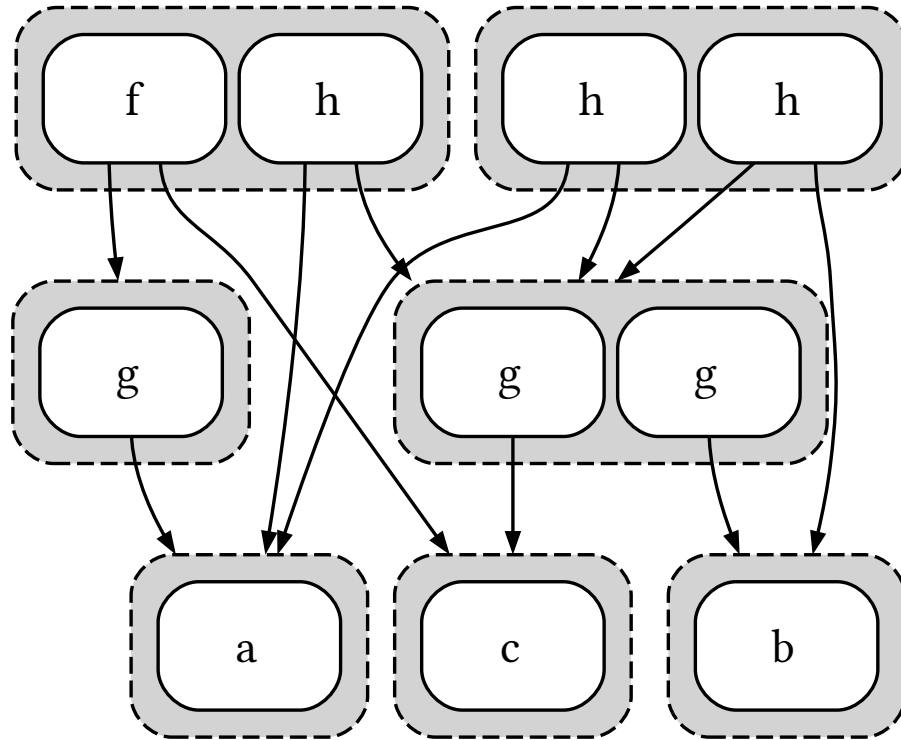
E-graphs are just certain kinds of rewrite systems

E-graphs as a rewrite system



$$\begin{aligned}
 a &\rightarrow e_a \\
 b &\rightarrow e_b \\
 c &\rightarrow e_c \\
 g(e_a) &\rightarrow e_1 \\
 g(e_b) &\rightarrow e_2 \\
 g(e_c) &\rightarrow e_2 \\
 &\dots
 \end{aligned}$$

E-graphs as a rewrite system



$$f(e_1, e_c) \rightarrow e_3$$

$$h(e_a, e_2) \rightarrow e_3$$

$$h(e_a, e_2) \rightarrow e_4$$

$$h(e_2, e_b) \rightarrow e_4$$

What congruence closure is doing

$$e_3 \leftarrow h(e_a, e_2) \rightarrow e_4$$

When we can rewrite in two different ways, we call it a **critical pair**.

Resolve critical pairs by merging: in this case, e_3 and e_4

AC closure

We need to find AC-critical pairs

$$c + b \leftarrow (a + a) + b \stackrel{=}{\underset{\{AC\}}{}} a + (a + b) \rightarrow a + b$$

So, we learn that $c + b = a + b$

AC closure

Loop:

- Find an AC-critical pair $t_1 \leftarrow s \rightarrow t_2$
- Take the equation $t_1 = t_2$ and **orient** it into a new AC-rewrite rule
 $t_1 \rightarrow t_2$ or $t_2 \rightarrow t_1$
- Interreduce the new rule with the old ones

Integrating theories and E-graphs: design choices

There are two ways to think about adding an AC-solver to an E-graph.

1. ‘Integrated’: Treat +-nodes as any other E-graph node, and scan for critical pairs between them
2. ‘Side-table’: Don’t store +-nodes in the normal E-graph at all. Instead, have a side table of AC-rewriting rules.
 - When you add e.g. $e_1 + e_2$ to the E-graph, create a new rewrite $e_1 + e_2 \rightarrow e_3$ for some fresh e_3
 - Whenever we discover a rewrite $e_j \rightarrow e_k$ in the AC-solver, feed it back to the original E-graph

Integrating theories and E-graphs

I chose option 2 for my current implementation. Indeed, I didn't even think too hard about the tradeoff, as it seemed obvious that these are equivalent in some sense.

The integrated approach

We could take a discovered equation $t_1 = t_2$ and introduce a new E-class id e and add

$$t_1 \rightarrow e$$

$$t_2 \rightarrow e$$

This is more or less the same as the side-table approach (I intent to give a proof in my thesis) and a bit more “E-graph like”, since we only ever have $f(\dots) \rightarrow e$ and never $f(\dots) \rightarrow g(\dots)$ in ordinary E-graphs.

Equivalent, but only if you pay attention

However, there is one advantage of the ‘side-table’ approach that might be accidentally missed in the integrated approach.

In the side-table approach, if we build the rewrite $e_1 + e_2 \rightarrow e_3$ and we have an existing rule $e_1 + e_2 + e_2 \rightarrow e_4$, we can compress the existing rule down to $e_2 + e_3 \rightarrow e_4$.

This is a standard step in textbook Buchberger’s algorithm.

Equivalent, but only if you pay attention

To make the integrated method equivalent, we need to do some *destructive rewriting* of the E-graph in a way that is maybe a little alien to the typical ways of working with E-graphs.

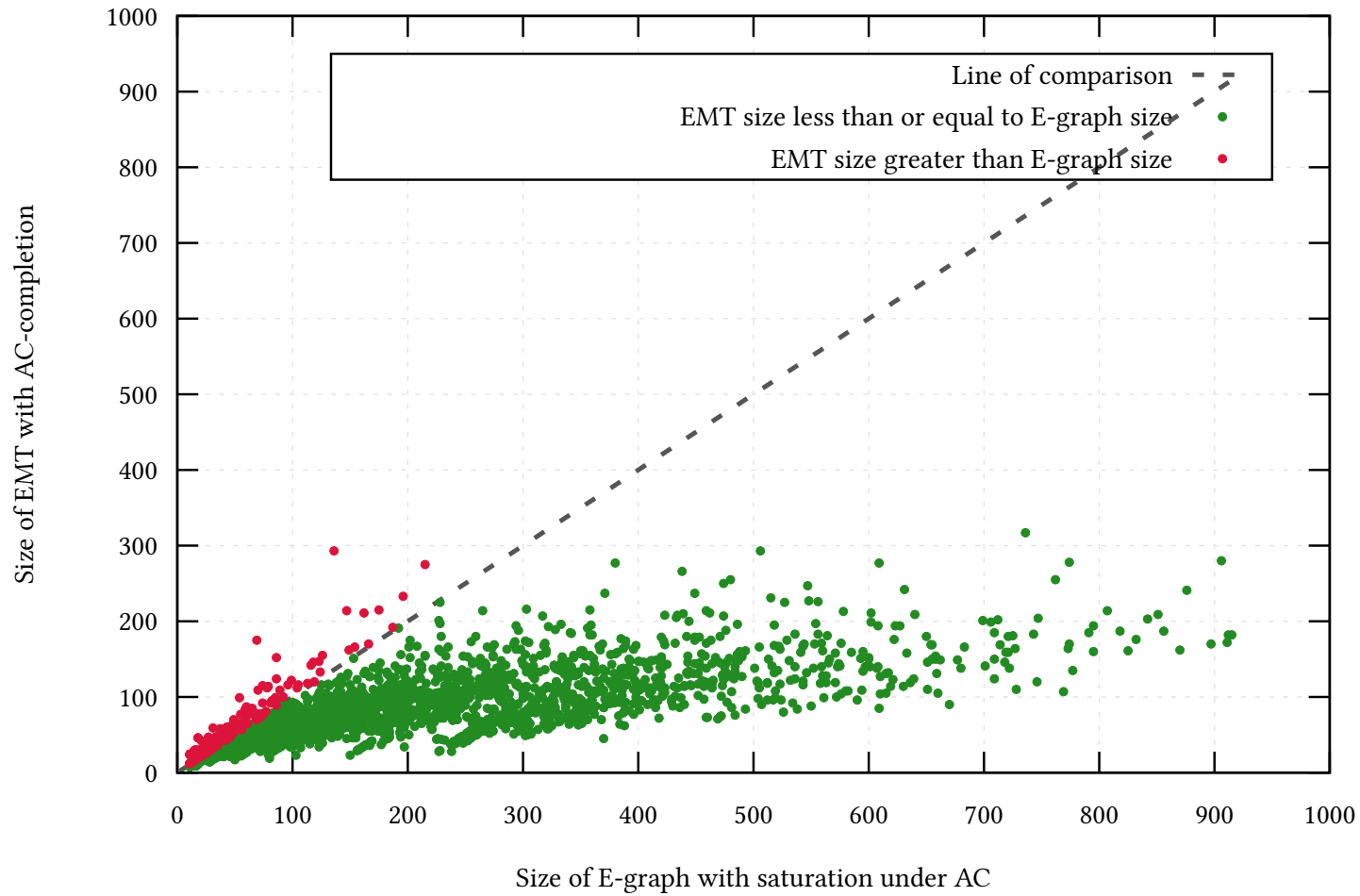
Results

Generate a bunch of random identities (e.g. $f(x) = h(x, f(x))$) and random initial equations.

Run to saturation, handling AC either via EMTs or via saturating on the AC identities

Measure the final size (size of everything, essentially)

Result



AC-closure and the wider world

AC-closure is just a form of **Buchberger's algorithm**, a well-known algorithm in algebra.

Lots of work has been done to optimize it.

Still, the problem is fundamentally ESPACE-complete.

EMTs for any T

Filling the “gap” is exactly equivalent to a well-known problem, the **word problem**

For just associativity, the word problem is undecidable! So, we couldn't just run “A-closure”: we would have to allow the option to run it up to some number of steps, with no guarantees it will ever be done.

Some word problems are much better than ESPACE:

- Abelian groups/ACUN: polynomial time
- Fields, vector spaces, etc: $O(n^{2.\text{something}})$

For AC + other identities (semirings, rings, modules, ...), often a variant of Buchberger's algorithm suffices.

T -closure

AC-closure has been developed by Tiwari and Kapur, among others, and between them they give a variety of variants to handle scalar multipliers, inverses, idempotency, etc.

Matching modulo AC

One tricky thing is cycles.

For instance, consider the equation $a + 0 = a$.

Then the pattern $a + x$ could match

- $a + 0$
- $a + (0 + 0)$

...

Matching via rewriting

Finding some set of minimal representatives of the set of matches is exactly the kind of thing our AC-rewriting system is made for!

Matching via rewriting: vanilla E-graphs

Consider the rules

$$f(e_1, e_1) \rightarrow e_3$$

$$g(e_3) \rightarrow e_1$$

and the pattern $f(g(x), g(x))$

Pattern compilation

$f(g(x), g(x))$ can be compiled to the “rewrite system”

$$g(x) \xrightarrow{M} s_1(x)$$

$$f(s_1(x), s_1(x)) \xrightarrow{M} s_2(x)$$

We have a bunch of match states $s_1, s_2, \dots$ and each match state carries a partial substitution.

Running the match system

$$e_1 \leftarrow g(e_3) \xrightarrow[M]{} s_1(e_3)$$

So, we ought to annotate e_1 with the match state $s_1(e_3)$

A lot like finding critical pairs!

Running the match system

If $e_1 \xrightarrow[M]{} s_1(e_3)$, then we have the “critical pair”

$$e_3 \leftarrow f(e_1, e_1) \xrightarrow[M]^* f(s_1(e_3), s_1(e_3)) \xrightarrow[M]{} s_2(e_3).$$

So,

$$e_3 \xrightarrow[M]{} s_2(e_3)$$

There can be multiple matches

So it can be the case that $t \xrightarrow{M} s_{i(\dots)}$ and $t \xrightarrow{M} s_{j(\dots)}$

AC-matching

Match $a + x$

against

$$\begin{aligned} a &\rightarrow e_a \\ e_a + 0 &\rightarrow e_a \end{aligned}$$

AC-matching

First “critical pair”: $e_a \leftarrow a \xrightarrow[M]{} s_1$ So $e_a \xrightarrow[M]{} s_1$

AC-matching

Second “critical pair”: $e_a \leftarrow e_a + 0 \xrightarrow{M} s_1 + 0 \xrightarrow{M} s_2(0)$

So $e_a \xrightarrow{M} s_2(0)$

AC-matching

But the matcher itself has a critical pair:

$$s + s_2(t) \xleftarrow{M} s + t + 0 \xrightarrow{M} s_2(s + t)$$

So we ought to add the rule $s_2(s + t) \rightarrow s + s_2(t)$

AC-matching

Then the match $s_2(0 + 0)$ would reduce to $0 + s_2(0)$ and thus to $s_2(0)$

Tentative conclusion/conjecture

AC-matching can be done in an (always terminating?) way by orienting rules to handle the otherwise potentially-infinite match space

Future work: Further considerations for AC-matching

Consider the pattern $x + y + z + w$. There are a lot of ways this can match against a term $a + b + c + d$, but we probably only want one of these matches.

So it is natural to consider patterns with **ordering constraints**: “match $x + y + z + w$ where $x \leq y \leq z \leq w$ ”

Conclusion

I started working on the AC-E-graph-matching problem and realized that without AC-closure, E-graphs modulo AC are not very useful.

See my upcoming thesis for more details, although the details of AC-matching for EMT(AC) are still rather sketchy.